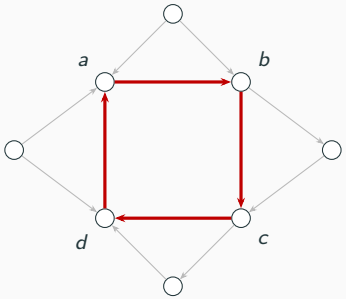
Counting 4-Cycles in Fully Dynamic Graphs

A Cyclic-Join Perspective

Speaker 1 Speaker 2 Speaker 3 Speaker 4

Algorithms seminar – mock PODS 2025

A motivating query: cyclic patterns in a transaction graph



accounts (nodes), transactions (arrows)

Account a pays b, b pays c, c pays d, d pays back to a – a 4-cycle in the transaction graph, and a classic pattern of interest in fraud detection.

Formally, this is a cyclic conjunctive query: four relations joined in a loop. The size of its answer is exactly the number of 4-cycles in the graph.

The same structure appears in many settings – citation loops, mutual-follow communities, recommendation cycles.

And the underlying graph is constantly evolving.

The fully dynamic setting

Every new transaction inserts an edge; deletions are equally permitted. After *every* update, the system must report the exact 4-cycle count.

Recomputing from scratch is prohibitively slow on large graphs. We require fast per-update maintenance.

Worst-case, not amortised

The cost we measure is the *slowest* update – not the average over a sequence.

Database systems are sensitive to tail latency: a single one-second update is a production incident, even if the mean is fast.

The target is therefore sharp: beat recompute-from-scratch under a per-update worst-case guarantee.

The landscape: three small patterns

Best known per-update maintenance cost, for each pattern:

Pattern	Shape	Best per-update cost
Triangle (3-cycle)		$\sqrt{m}$
4-clique		m
4-cycle		$m^{2/3}$

- Triangles and 4-cliques settle at clean, natural exponents.
- 4-cycles sit at an unusual $2/3$ – neither square root nor linear.
- That $2/3$ bound stood as the state of the art for over a decade.

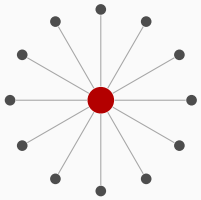
Where does the $m^{2/3}$ bound come from?

Every algorithm in this space exploits the same observation: vertex degrees are highly unequal.

- Few high-degree vertices (e.g. banks, large merchants) – each expensive to process.
- Many low-degree vertices – each cheap individually.

Counting 4-cycles separately for the two classes, and choosing the threshold so the two costs balance, yields exactly $m^{2/3}$.

This balance is the natural ceiling for any combinatorial algorithm operating one edge at a time.



high-degree: one vertex, many edges

low-degree: many vertices, few edges each

The 2025 result of Assadi and Shah

Assadi and Shah (PODS 2025) finally broke the wall, achieving worst-case update time $m^{2/3-\varepsilon}$ for a constant $\varepsilon > 0$.

The unexpected ingredient: fast matrix multiplication.

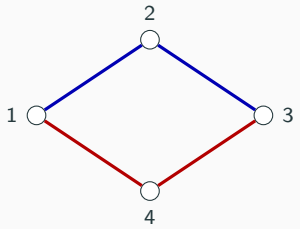
Why is that surprising?

- Fast matrix multiplication is a bulk primitive – efficient only when applied to two large matrices at once.
- Our setting is streaming – edges arrive one at a time.
- Folklore held that these two are incompatible.

Reconciliation: phases

The update stream is organised into phases. During each phase, the algorithm answers queries against a previously computed matrix product while the next product is precomputed in the background, amortised across the phase. At the phase boundary, the two products swap.

Why matrix multiplication?



Two 2-hop paths from 1 to 3
$(1 \to 2 \to 3 + 1 \to 4 \to 3)$
= one 4-cycle.

A 4-cycle is two 2-hop paths sharing the same pair of endpoints.

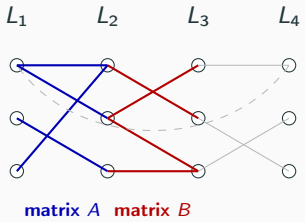
Counting 4-cycles therefore reduces to: for every pair of vertices (u, v), how many 2-hop paths connect them?

The answer is a table indexed by pairs of vertices – one entry per pair.

Computing this table is precisely a matrix multiplication.

The new algorithm uses *fast* matrix multiplication for that step – the origin of the asymptotic speedup.

The cyclic-join lens: where the speedup originates



$A \cdot B$ = the 2-path table from slide 7.

Each vertex of the 4-cycle from slide 7 plays one of four roles: position 1, 2, 3, or 4.

Layer L_i collects every vertex viewed as a candidate for role i – conceptually, four copies of the vertex set.

A: edges between role-1 and role-2 candidates.
B: edges between role-2 and role-3 candidates.

Then $A \cdot B$ counts, for each pair $(1, 3)$, the number of role-2 intermediates – exactly the top half of slide 7's 4-cycle.

Fast matrix multiplication evaluates this product strictly faster than the naive approach – the shortcut that breaks $m^{2/3}$.

One matrix step, supported by scaffolding

We have located the matrix step. But a fundamental tension remains:

Bulk vs. streaming

Fast matrix multiplication is bulk: it requires two complete matrices, multiplied at once.

The problem is streaming: edges arrive one at a time.

Every remaining component of the algorithm exists to bridge that gap:

- Degree partitioning $\rightarrow$ control matrix size and shape.
- Chunked maintenance $\rightarrow$ amortise the bulk cost across many edges.
- Phases $\rightarrow$ keep a precomputed product ready while the next is being assembled.

All of this complexity serves a single purpose: render a bulk matrix multiplication compatible with a streaming workload.

Fast matrix multiplication is essential, not incidental

Could the speedup arise from a different part of the machinery? We verified that it cannot.

We analysed the algorithm's constraint system at the conjectural optimum – the best possible matrix-multiplication exponent.

The conclusion is sharp:

Remove the fast matrix multiplication step and the entire speedup vanishes; the bound collapses back to the prior $m^{2/3}$.

Fast matrix multiplication is the load-bearing step. The surrounding scaffolding exists to make it usable in a streaming setting.

Implementation and empirical setup

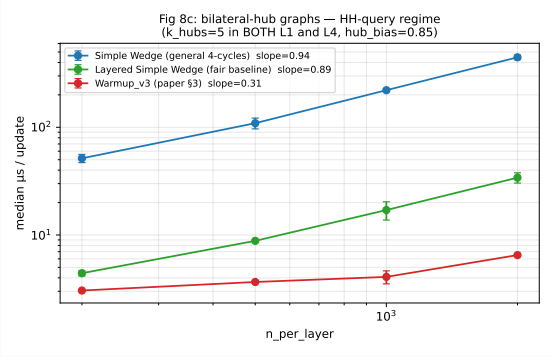
Theory establishes that the matrix step is essential. The empirical question: on which inputs is the surrounding machinery actually worth its complexity?

Three components, all in Python:

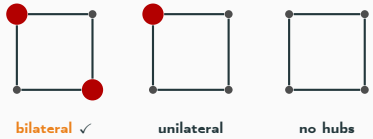
- Algorithm. A faithful implementation of the new algorithm's structure – four layers, degree thresholds, chunked update maintenance, and the central matrix-multiplication step.
- Baseline. The natural simpler approach – wedge-counting on the same four-layered graph, without the class-routing machinery.
- Test harness. A graph generator with controllable structure: random graphs, one-sided high-degree, two-sided high-degree, or uniformly low-degree.

Both algorithms run on identical graphs and identical update streams. We characterise the regime in which the new algorithm wins, not its asymptotic exponent.

Where the algorithm wins: bilateral high-degree structure



The new algorithm outperforms the baseline exactly when the graph has high-degree vertices on opposite sides of the cycle:



On the other regimes, the baseline matches the new algorithm closely – the extra machinery becomes overhead.

The algorithm is therefore not a generic accelerant. It targets a specific structural regime in which fast matrix multiplication is worth setting up.

Takeaway

Two perspectives converge on the same conclusion:

- Theoretical. Only one step is load-bearing – the fast matrix multiplication.
- Empirical. Only one regime requires the machinery – bilateral high-degree.

Both indicate that the surrounding machinery exists to support one well-conditioned matrix multiplication.

Open directions:

- Larger cycles – k-cycles beyond 4.
- Online detection of the bilateral regime, with fallback to the baseline.
- Closed-form analysis at *current* matrix-multiplication exponents.

More broadly, a clean lens on the tension between bulk and streaming computation in dynamic data systems.

Thank you.

Questions?